Подбор гиперпараметров

Часть 1

Лабораторная работа 6

Подбор гиперпараметров. Визуализация дерева решений

- Продолжаем работать в блокноте лабораторной 5 (или делаем копию)

Создадим модель DecisionTree

```
[14] from sklearn.tree import DecisionTreeClassifier
     dtf = DecisionTreeClassifier(random_state=0)
     dtf.fit(X_train,Y_train)
     pred_1=dtf.predict(X_test)
```

Построим матрицу ошибки

```
[15] df1=pd.crosstab(Y_test,pred_1)
     df1
```

```
from sklearn.metrics import accuracy_score,precision_score,recall_score
print('accuracy',accuracy_score(Y_test,pred_1))
print('precision',precision_score(Y_test,pred_1))
print('recall',recall_score(Y_test,pred_1))
```

```
accuracy 0.8244274809160306
precision 0.8
recall 0.6808510638297872
```

Подбор гиперпараметров. Визуализация дерева решений

- Продолжаем работать в блокноте лабораторной 5 (или делаем копию)

```
[17] from sklearn.metrics import classification_report
     print(classification_report(Y_test,pred_1))
```

```
              precision    recall  f1-score   support

           0       0.84      0.90      0.87        84
           1       0.80      0.68      0.74        47

    accuracy                           0.82       131
   macro avg       0.82      0.79      0.80       131
weighted avg       0.82      0.82      0.82       131
```

Подбор гиперпараметров. Визуализация дерева решений

Выводим важность признаков в этой модели

```
[19] dtf.feature_importances_
```

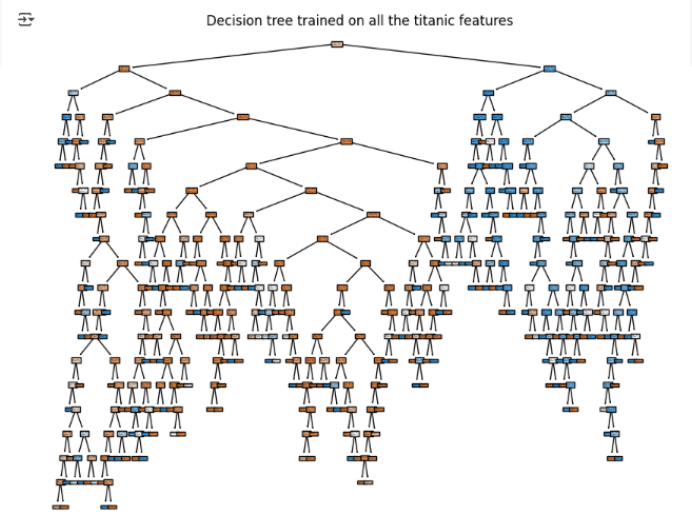
0 сек.

```
array([0.06064187, 0.49788065, 0.16924894, 0.02386817, 0.02637836,
       0.20814403, 0.01383798])
```

Подбор гиперпараметров. Визуализация дерева решений

Строим дерево

```
[20] from sklearn.tree import plot_tree
     plt.figure(figsize=(10,8))
     plot_tree(dtf, filled=True)
     plt.title("Decision tree trained on all the titanic features")
     plt.show()
```



Подбор гиперпараметров. Визуализация дерева решений

подберём оптимальные гиперпараметры

class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0, monotonic_cst=None)

Важные параметры: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[21]
```
param_grid = {
    'splitter': ['best', 'random'],
    'max_features': [None, 'sqrt', 'log2'],
    'max_depth' : [4,5,6,7,8,10],
    'criterion' :['gini', 'entropy','log_loss']
}
```

[22]
```
from sklearn.model_selection import GridSearchCV
CV_dtf = GridSearchCV(estimator=dtf, param_grid=param_grid, cv= 5)
CV_dtf.fit(X, Y)
```

```
GridSearchCV
best_estimator_:
DecisionTreeClassifier
DecisionTreeClassifier
```

Подбор гиперпараметров. Визуализация дерева решений

Узнаем найденные параметры

```
[23] CV_dtf.best_params_
```

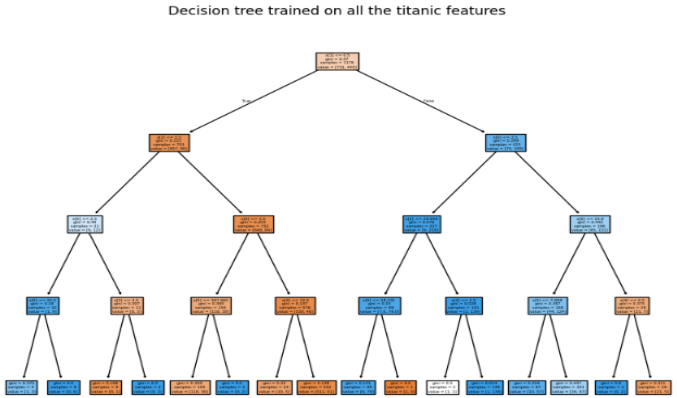
```
{'criterion': 'gini', 'max_depth': 4, 'max_features': None, 'splitter': 'best'}
```

```
[24] CV_dtf.best_score_
```

```
np.float64(0.8638823082097629)
```

Упражнения

1. Попробуйте поменять параметр разбиения для перекрёсной проверки cv=10 или cv=12
2. Постройте модель DecitionTree с найденными оптимальными параметрами: обучите на X_train, проверьте на X_test
3. Постройте матрицу ошибки + метрики
4. Визуализируйте дерево решений с оптимальными параметрами



Упражнения

- Выполните подбор оптимальных гиперпараметров для RandomForest
- Выполните подбор оптимальных гиперпараметров для других моделей
- Постройте ROC-кривые для моделей с оптимальными гиперпараметрами
- Выполните подбор оптимальных гиперпараметров для моделей на уменьшенном и увеличенном наборе признаков. Проанализируйте результаты

Упражнения

- Рассмотрите пример работы randomizedsearch для ирисов Фишера (сл. 11- 15)
- Используйте randomized search для ваших моделей на датасете Титаника

Пример использования randomized search and grid search для ирисов Фишера

```
from time import time

import numpy as np
import scipy.stats as stats

from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

# get some data
X, y = load_digits(return_X_y=True, n_class=3)

# build a classifier
clf = SGDClassifier(loss="hinge", penalty="elasticnet", fit_intercept=True)
```

Взят из документации

Пример использования randomized search and grid search

```
# Utility function to report best scores
def report(results, n_top=3):
    for i in range(1, n_top + 1):
        candidates = np.flatnonzero(results["rank_test_score"] == i)
        for candidate in candidates:
            print("Model with rank: {0}".format(i))
            print(
                "Mean validation score: {0:.3f} (std: {1:.3f})".format(
                    results["mean_test_score"][candidate],
                    results["std_test_score"][candidate],
                )
            )
            print("Parameters: {0}".format(results["params"][candidate]))
            print("")
```

Пример использования randomized search and grid search

```
# specify parameters and distributions to sample from
param_dist = {
    "average": [True, False],
    "l1_ratio": stats.uniform(0, 1),
    "alpha": stats.loguniform(1e-2, 1e0),
}

# run randomized search
n_iter_search = 15
random_search = RandomizedSearchCV(
    clf, param_distributions=param_dist, n_iter=n_iter_search
)

start = time()
random_search.fit(X, y)
print(
    "RandomizedSearchCV took %.2f seconds for %d candidates parameter settings."
    % ((time() - start), n_iter_search)
)
report(random_search.cv_results_)
```

Пример использования randomized search and grid search

```
# use a full grid over all parameters
param_grid = {
    "average": [True, False],
    "l1_ratio": np.linspace(0, 1, num=10),
    "alpha": np.power(10, np.arange(-2, 1, dtype=float)),
}

# run grid search
grid_search = GridSearchCV(clf, param_grid=param_grid)
start = time()
grid_search.fit(X, y)

print(
    "GridSearchCV took %.2f seconds for %d candidate parameter settings."
    % (time() - start, len(grid_search.cv_results_["params"]))
)
report(grid_search.cv_results_)
```

Пример использования randomized search and grid search (результат)

```
RandomizedSearchCV took 4.33 seconds for 15 candidates parameter settings.
Model with rank: 1
Mean validation score: 0.989 (std: 0.007)
Parameters: {'alpha': np.float64(0.306366480190733), 'average': False, 'l1_ratio': np.float64(0.23280367025289272)}

Model with rank: 2
Mean validation score: 0.987 (std: 0.010)
Parameters: {'alpha': np.float64(0.019502944439424907), 'average': False, 'l1_ratio': np.float64(0.08475435803676135)}

Model with rank: 3
Mean validation score: 0.985 (std: 0.015)
Parameters: {'alpha': np.float64(0.020832876936895228), 'average': False, 'l1_ratio': np.float64(0.33174947055743964)}

GridSearchCV took 13.40 seconds for 60 candidate parameter settings.
Model with rank: 1
Mean validation score: 0.993 (std: 0.009)
Parameters: {'alpha': np.float64(1.0), 'average': False, 'l1_ratio': np.float64(0.0)}

Model with rank: 2
Mean validation score: 0.989 (std: 0.018)
Parameters: {'alpha': np.float64(0.01), 'average': True, 'l1_ratio': np.float64(0.111111111111111)}

Model with rank: 2
Mean validation score: 0.989 (std: 0.014)
Parameters: {'alpha': np.float64(0.1), 'average': False, 'l1_ratio': np.float64(0.111111111111111)}
```